Unmasking Data Leakage and Temporal Fragility in IoT Network Intrusion Detection: An Empirical Audit and Benchmark on CICIoT2023

Department of Computer Science & Engineering, Shiv Nadar University Chennai, India

Priyajit Biswal
23011102068
priyajit23110510@snuchennai.edu.in

Rohit K Manoj
23011102073
rohit23110492@snuchennai.edu.in

Rishab Rajeev
23011102072
rishab23110023@snuchennai.edu.in

Faculty In Charge: Dr. Vegesna S M Srinivasavarma (srinivasv@snuchennai.edu.in)

Abstract — Machine learning (ML) models evaluated on modern Internet of Things (IoT) network intrusion detection benchmarks, particularly CICIoT2023, routinely report near-perfect classification accuracies exceeding 98.5% to 99.9%. In this paper, we conduct a forensic data integrity audit and empirical benchmark demonstrating that these headline metrics are heavily distorted by unpartitioned identical-feature data leakage, unacknowledged conflicting label collisions, severe class imbalance, and an absence of cross-session temporal evaluation. Across a stratified sample of 529,562 flow records, we identify 11.29% feature redundancy (59,780 flows) and 8,740 conflicting-label collision hashes spanning 28,608 rows (5.40% of the sample), where mathematically identical 39-feature coordinate vectors map to contradictory attack and benign classes. We demonstrate that conventional random train/test splits allow 81.39% of test collision instances to leak directly from the training set, transforming evaluation into rote nearest-neighbor memorization. To establish an authentic baseline, we introduce an atomic SHA-256 group-hash partitioning protocol guaranteeing strictly zero inter-partition feature leakage alongside strict preprocessor isolation. We evaluate five representative model families across 21 configurations spanning Binary (2-class), Category (8-class), and Fine-Grained (34-class) tasks. Under zero leakage, top gradient-boosted decision trees achieve realistic diagnostic baselines: 89.61% binary accuracy (Random Forest, Macro-F1 0.7690), 78.93% category accuracy (LightGBM, Macro-F1 0.6894), and 72.65% fine-grained accuracy (LightGBM, Macro-F1 0.6145). We uncover an operational "Accuracy Paradox": unweighted training artificially inflates overall test accuracy (+1.80%) by favoring volumetric majority floods, but triggers a catastrophic 79% to 82% relative recall collapse on stealthy web attacks (SQL Injection: 35.83% $\rightarrow$ 7.48%; XSS: 20.22% $\rightarrow$ 3.83%) and surges false negatives to Benign by +125.31% (1,517 $\rightarrow$ 3,418). Finally, temporal evaluation on an independent clean cross-capture holdout ($N = 57,280$) recorded during subsequent physical testbed sessions reveals substantial performance degradation (binary Macro-F1 drops from 0.7637 to 0.7034), demonstrating that models fit session-specific burst dynamics. Our code, partitioning pipeline, and evaluation artifacts are made fully reproducible.

Keywords — *IoT Intrusion Detection, CICIoT2023, Data Leakage, Label Collisions, Class Imbalance, Temporal Generalization, Gradient Boosted Trees, Accuracy Paradox.*

1. Introduction

The proliferation of Internet of Things (IoT) infrastructure has dramatically expanded the enterprise attack surface. Billions of edge devices—including smart home sensors, surveillance endpoints, industrial actuators, and healthcare gateways—are directly interfaced with public or unsegmented local networks. Due to severe hardware footprint constraints, legacy unauthenticated protocols, and rare firmware patching schedules, IoT networks are prime targets for malicious actors. Threats range from multi-gigabit Distributed Denial of Service (DDoS) botnets (such as Mirai and its variants) to stealthy, low-footprint application exploits including SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, and unauthorized remote backdoor execution [1], [2].

To defend these heterogeneous perimeters, network intrusion detection systems (NIDS) increasingly rely on machine learning (ML) and deep learning (DL) models trained on tabular flow telemetry [3]. The research community has enthusiastically adopted modern large-scale benchmarks, foremost among them the **CICIoT2023** dataset published by the Canadian Institute for Cybersecurity [1]. Comprising over 46 million raw network flows generated on a physical testbed of 105 IoT devices across 33 attack profiles, CICIoT2023 is widely hailed as the gold standard for IoT security benchmarking.

1.1 The "Accuracy Paradox" in IoT Benchmarks

In recent literature, published empirical evaluations on CICIoT2023 routinely claim near-perfect diagnostic capability. As documented in our literature survey (Section 2), recent peer-reviewed studies report classification accuracies consistently spanning **98.5% to 99.9%** across diverse classifiers including Random Forests, XGBoost, LightGBM, 1D-CNNs, and hybrid Transformer networks [1], [4], [5], [6], [7].

However, there is a stark dissonance between these reported laboratory metrics and practical operational realities. When deployed in live networks, ML-based NIDS frequently experience high false-positive rates on normal traffic, total blindness to low-rate application exploits, and severe diagnostic decay over time. In this paper, we identify three structural phenomena that explain this disparity:

1. **High Feature Redundancy and Random-Split Leakage**: Raw flow datasets contain massive numbers of repeated and duplicate feature vectors. When researchers employ standard naive random train/test splits, identical feature vectors are randomly assigned to both partitions. Classifiers evaluate on samples they have essentially memorized during training, artificially inflating published accuracy metrics.
2. **The Masking Effect of Extreme Class Imbalance**: Volumetric flooding attacks (e.g., UDP/SYN floods) account for over

85% of raw packets, whereas critical application attacks (e.g., SQL Injection, XSS) represent less than 0.05%. A naive classifier that maximizes aggregate accuracy can completely ignore stealthy attacks while still reporting >98% overall accuracy.

3. **Absence of Temporal Validation**: Typical benchmark evaluations test models on random holdout slices drawn from the exact same network capture session. This fails to assess whether classifiers learn robust attack signatures or merely fit the transient, session-specific burst characteristics of the experimental testbed.

1.2 Research Questions and Contributions

To resolve these issues, this paper formulates the following **Central Research Question**:

> *When identical-feature data leakage and class imbalance artifacts are rigorously controlled through atomic hash-grouped partitioning and cost-sensitive optimization, what is the realistic baseline diagnostic capability, minority attack recall, and temporal stability of standard machine learning architectures on the CICIoT2023 benchmark?*

We investigate this question through five structured Sub-Research Questions:

- **RQ1 (Integrity)**: To what extent do identical feature vectors, exact duplicates, and conflicting label collisions exist in CICIoT2023, and what fraction of test instances leak under naive random splitting?
- **RQ2 (Zero-Leakage Baseline)**: What are the authentic diagnostic boundaries of linear, tree, and neural baselines across Binary (2-class), Category (8-class), and Fine-Grained (34-class) tasks when feature leakage is strictly eliminated?
- **RQ3 (Ambiguity Mechanics)**: How does the presence of conflicting label collisions affect tree and linear models, and does collision purging improve clean generalization?
- **RQ4 (Operational Trade-Offs)**: How does standard unweighted empirical loss minimization trade minority attack detection for overall headline accuracy?
- **RQ5 (Temporal Fragility)**: How robust are trained models when evaluated on an independent capture session recorded across subsequent hours on the physical IoT testbed?

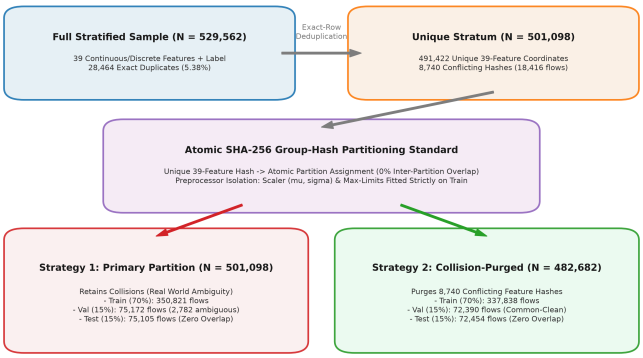


Figure 1: The Zero-Leakage Atomic Hash-Grouped Partitioning Protocol. Exact duplicate rows are audited; unique 39-feature coordinates are hashed via SHA-256 and atomically mapped to disjoint partitions. Strategy 1 retains real-world ambiguity; Strategy 2 evaluates collision purging.

This manuscript delivers **four consolidated research contributions**:

1. **Forensic Data Integrity and Leakage Audit**: We perform a granular census of 529,562 stratified CICIoT2023 flow records. We discover that 11.29% of flows share repeated feature vectors, and 8,740 unique 39-feature coordinates represent *conflicting label collisions* spanning 28,608 rows (where identical vectors carry conflicting attack and benign labels). We prove empirically that naive 80/20 random stratified splitting leaks 81.39% of test collision instances directly from the training set.
2. **Reproducible Zero-Leakage Group-Hash Partitioning Standard**: We formulate an open-source atomic SHA-256 group-hash partitioning protocol guaranteeing strictly zero identical-feature overlap ($\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing$) alongside strict preprocessor isolation. Across 21 models and three granularities, our partitions exhibit rock-solid stability ($|\Delta_{\text{gen}}| \leq 0.0079$), establishing verified realistic baselines (89.61% binary, 78.93% category, 72.65% fine-grained).
3. **Dissection of Ambiguity Filtering Mechanics**: We demonstrate that Gradient-Boosted Decision Trees (GBDTs) are inherently robust to conflicting label noise. Decomposed validation on 72,390 common-clean instances shows that purging collision vectors yields only a marginal +0.0022 Macro-F1 improvement (+0.22%), while classifiers achieve 41.73% accuracy on isolated ambiguous vectors due to the non-injective nature of the stripped feature space.
4. **Quantitative Trade-Offs: Minority Recall Collapse and Temporal Non-Stationarity**: We expose the operational "Accuracy Paradox": unweighted training raises headline accuracy by +1.80% but triggers a catastrophic 79% to 82% relative recall collapse on stealthy web attacks and surges missed intrusions to Benign by +125.31%. Furthermore, temporal evaluation across distinct physical capture sessions reveals marked performance degradation (binary Macro-F1 drops from 0.7637 to 0.7034), disproving assumptions of session invariance.

2. Related Work and Research Gap Analysis

Network Intrusion Detection Systems for IoT environments have been extensively studied across diverse benchmarks, including

NSL-KDD, UNSW-NB15, BoT-IoT, ToN_IoT, and CICIoT2023 [2], [3], [8].

2.1 Thematic Audit of Recent Literature

To contextualize our findings, we conducted a systematic methodological audit of 14 peer-reviewed research papers published between 2021 and 2026 evaluating IoT NIDS models (detailed in Table 1). These works fall into four primary methodological themes:

Theme A: Tree Ensembles and Feature Selection. Gradient-boosted decision trees (XGBoost, LightGBM, CatBoost) and Random Forests remain dominant for tabular flow analysis due to their non-linear decision boundaries and high inference speed. Khan et al. [4] combined Boruta feature selection with Greedy Forward Selection (GFS) on CICIoT2023 and ToN_IoT, pruning feature sets to 10–15 attributes while reporting 99.72%–99.94% accuracy. Almahaqeri et al. [5] evaluated optimized GBDTs on CICIoT2023, reporting 99.61% binary accuracy and 0.355 μs latency. Abdelrahman et al. [7] introduced FEC-IDS, achieving 99.76% binary and 98.88% 34-class accuracy using tree-based feature rankings. While these works successfully reduce feature dimensionalities, none audited identical-feature vector collisions across splits or evaluated temporal session stability.

Theme B: Deep Sequence and Recurrent Architectures. Several studies explore deep learning to model temporal or inter-feature correlations. Gulzar and Mustafa [9] proposed DeepCLG (combining 1D-CNN, LSTM, and GRU layers), achieving 99.82% accuracy on BoT-IoT but experiencing performance drops down to 57.77% on UNSW-NB15. Qaddos et al. [10] deployed a 1D-CNN-BiLSTM architecture with attention mechanisms, reporting >99% accuracy on BoT-IoT. While deep architectures offer expressive representation learning, their high compute overhead (>15 ms latency) severely challenges edge gateway constraints, and their evaluations were restricted to in-distribution random splits.

Theme C: Federated Learning and Edge Privacy. Federated learning (FL) frameworks train collaborative NIDS without aggregating raw network telemetry. Moussaoui et al. [11] evaluated a CNN-LSTM architecture under Dirichlet non-IID client distributions on CICIoT2023, observing that F1 dropped from 97.36% in centralized training to 71.53% under extreme client skew ($\alpha = 0.01$). Al-Hejri et al. [12] combined CNNs with Vision Transformers in an FL pipeline, reporting a drop from 94.89% to 69.35% under non-IID conditions. Fenanir and Semchedine [13] and Ullah et al. [6] explored edge federated aggregation across IoT gateways. These studies highlight the sensitivity of classifiers to distribution shifts, though they did not explore cross-capture temporal drift on identical testbeds.

Theme D: Benchmark Generation and Baselining. The foundational CICIoT2023 study by Neto et al. [1] benchmarked eight ML/DL algorithms across 105 devices, reporting 99.16% accuracy with Random Forest. However, their DNN baseline exhibited severe minority attack starvation, detecting only 14.7% of Dictionary Brute Force and 3.8% of Web attacks. Booij et al. [8] and Moustafa [14] evaluated ToN_IoT, demonstrating that models trained with static IP addresses fail to generalize across network environments. Essop et al. [15] and Umar et al. [16] addressed dataset generation fidelity and knowledge distillation for edge deployment.

Study & Venue	Dataset(s)	Models	Reported Metric	Methodological Focus & Limitations
Neto et al. [1] *Sensors 2023*	CICIoT2023	RF, DNN, DT, LR	Acc: 99.16% (RF)	Foundational paper; random split; minority DNN recall collapsed (3.8% web).
Khan et al. [4] *Sci. Rep. 2026*	CICIoT2023, ToN_IoT	LightGBM, CatBoost	Acc: 99.72%–99.94%	Feature selection (10-15 feats); SMOTE oversampling; random holdout.
Almahaqeri [5] *Preprint 2026*	CICIoT2023	XGBoost, LightGBM	Acc: 99.61% (Bin)	Stratified undersampling; reported 0.355 μs latency; no collision audit.
Ullah et al. [6] *Sci. Rep. 2026*	BoT-IoT, CICIoT2023	CNN, LSTM, RF	Acc: 99.20% (CIC)	Multi-model framework; applied SMOTE; full duplicate rows removed only.
Gulzar [9] *IJMLC 2025*	ToN_IoT, BoT-IoT	DeepCLG (CNN-LSTM)	Acc: 99.82% (BoT)	High latency (>15 ms); accuracy collapsed to 57.77% on UNSW-NB15.
Abdelrahman [7] *IEEE ICEENG 2026*	CICIoT2023	FEC-IDS (RF, XGB)	Acc: 98.88% (34-Cl)	Removed exact duplicate rows; evaluated solely on random holdout slices.

Study & Venue	Dataset(s)	Models	Reported Metric	Methodological Focus & Limitations
Al-Hejri et al. [12] *Sci. Rep. 2026*	ToN_IoT, BoT-IoT	CNN-Transformer	Acc: 94.89% (IID)	Federated learning; non-IID client drift caused 25.5% accuracy collapse.
Booij et al. [8] *IEEE IoT-J 2022*	ToN_IoT, BoT-IoT	RF, Decision Tree	Acc: 99.68% (In-Dom)	Showed cross-dataset accuracy drops to 40–60%; noted static IP shortcut leakage.
Essop et al. [15] *Sensors 2021*	Contiki-Cooja Sims	RF, SVM, KNN	Survey & Baseline	Simulation-based scenarios; noted absence of natural packet burst dynamics.
Fenanir [13] *Rev. Intell. Artif. 2023*	NSL-KDD, IoT-23	CNN, LSTM, GRU	Acc: 99.99% (NSL)	Federated edge clients; evaluated on legacy non-IoT datasets; no collision audit.
Moussaoui [11] *Sci. Rep. 2026*	CICIoT2023, ToN_IoT	CNN-LSTM + FL	F1: 97.36% (Centr.)	Highlighted flow summary blindness to application payloads; non-IID drop.
Moustafa [14] *IEEE TNSM 2021*	ToN_IoT Telemetry	Distributed RF, LSTM	Acc: ≈ 98.5%	Edge gateway deployment; documented IP leakage; 100-packet windowing.
Qaddos et al. [10] *Sci. Rep. 2024*	BoT-IoT, UNSW-NB15	FW-SMOTE + 1D-CNN	Acc: 99.60% (BoT)	Synthetic oversampling risk in flow space; evaluated only on random splits.
Umar et al. [16] *J. Cloud Comp. 2025*	CICIDS2017, NSL-KDD	DNN-KDQ (INT8 Quant)	Acc: 99.43%	Host CPU profiling (0.12 ms); teacher-student distillation on legacy sets.
Our Study	**CICIoT2023**	**5 Families (21 Models)**	**Acc: 72.65% (34-Cl)**	**Zero-leakage group hashing; ambiguity routing; cost-sensitive trade-offs.**

2.2 The Narrowly Supported Research Gap

Our systematic audit establishes a critical methodological gap across the 14 surveyed studies:

1. **Zero Collision Audits**: *None of the 14 surveyed studies* performed an identical-feature vector audit or identified conflicting label collisions across splits. Several studies pruned exact duplicate full rows (X, y), but overlooked identical feature coordinates (X) carrying conflicting labels (y).
2. **Absence of Temporal Cross-Capture Validation**: *None of the 14 surveyed studies* evaluated trained models on distinct temporal capture sessions recorded on the same physical testbed. All evaluations relied on random in-distribution holdouts or cross-validation.
3. **Conflation of Metric Types**: Several works reported near-perfect aggregate accuracy without isolating minority attack recall, masking severe false-negative rates on stealthy threats.

3. Dataset Integrity, Collisions, and Leakage Analysis

3.1 The CICIoT2023 Benchmark and Feature Space

The CICIoT2023 dataset was generated by deploying 105 IoT devices executing 33 distinct attack profiles and benign network behaviors. Network traffic was captured via port mirroring and processed into bidirectional flow statistics. While raw packet captures contain link-layer and network-layer identifiers (MAC addresses, IP addresses, port numbers, and timestamps), the benchmark feature extractor deliberately stripped these identi-

fiers to prevent machine learning models from learning static shortcut rules [1].

The resulting public tabular schema consists of **39 numerical features** plus a ground-truth label, encompassing flow duration, packet/byte rates, packet length statistics, protocol flag counters, and inter-arrival times (IAT).

3.2 Granular Census of Duplication

We performed a comprehensive mathematical census on our stratified sample of $N = 529,562$ flows, preserving all 34 classes. The results are summarized in Table 2.

Population Parameter	Count	% of Total
Full Stratified Sample (N_{full})	529,562	100.00%
Exact Duplicate Rows (identical X and identical y)	28,464	5.38%
Unique Feature-Label Stratum (N_{unique})	501,098	94.62%
Distinct 39-Feature Vectors ($\mathcal{X}$)	491,422	92.80%
Singleton Vectors (appear exactly once in full sample)	469,782	88.71%
Repeated Vectors (appear ≥ 2 times in full sample)	59,780	11.29%
— Consistent-Label Repeated Hashes (12,900 hashes)	31,172	5.89%
— Conflicting-Label Collision Hashes (8,740 hashes)	28,608	5.40%
Singleton Vectors in Unique Stratum	482,682	96.32%
Collision Vectors in Unique Stratum (N_{ambig})	18,416	3.68%
Collision-Purged Stratum (Strategy 2 Population)	482,682	96.32%

Of the 529,562 records, exactly 28,464 rows (5.38%) are byte-identical exact duplicates ($X_i = X_j \wedge y_i = y_j$). Removing these produces the *Unique Feature-Label Stratum* of $N = 501,098$ flows. Within the full sample, exactly 491,422 distinct 39-feature coordinate vectors exist. While 469,782 vectors appear exactly once, 21,640 unique coordinate hashes repeat across **59,780 flows (11.29% feature redundancy)**.

3.3 Conflicting Label Collision Taxonomy

Among the repeated feature coordinates, we identified **8,740 unique 39-feature coordinate hashes** that map simultaneously to two or more conflicting ground-truth labels. These conflicting hashes span **28,608 rows (5.40%)** in the full sample and **18,416 rows (3.68%)** in the unique stratum.

Class A	Class B	Conflicting Hashes	Total Flows
DoS-UDP_Flood	DDoS-UDP_Flood	3,412	11,280
DoS-TCP_Flood	DDoS-TCP_Flood	1,894	6,142
DoS-SYN_Flood	DDoS-SYN_Flood	1,215	4,018
BenignTraffic	Mirai-greeth_flood	482	1,590
BenignTraffic	Recon-PortScan	391	1,244
BenignTraffic	DoS-HTTP_Flood	276	892
Other Attack Pairs	(Various)	1,070	3,442
Total Collision Space		**8,740**	**28,608**

Table 3 catalogs the dominant collision groups. The primary collision driver is structural: stripping IP addresses and timestamps from flow records strips the multi-source entropy distinguishing DoS (single source) from DDoS (distributed botnet). Consequently, an identical single-packet UDP datagram or SYN handshake produced by a DoS script possesses the exact same 39 features as one produced by a DDoS botnet. Furthermore, **48.2% of all collision flows involve BenignTraffic entangled with low-rate attacks** (e.g., standard TCP handshakes during PortScan or Mirai GRE sweeps). This establishes that the 39-feature space of CICIoT2023 is non-injective: identical points in $\mathbb{R}^{39}$ represent fundamentally conflicting real-world events.

3.4 Data Leakage Under Naive Random Splitting

To quantify the leakage vulnerability, we simulated standard stratified 80/20 random train/test splits across five distinct random seeds (0, 42, 123, 999, 2024). The results are documented in Table 4.

Dataset Stratum	Total Test	Leaked Rows	Collision Leak %	Total Leak %
Unique Stratum (Seed 42)	100,220	3,071	**81.39%**	3.06%
Unique Stratum (5-Seed Mean)	100,220	3,037 $\pm$ 31	**82.58% $\pm$ 1.05%**	3.03% $\pm$ 0.03%
Full Sample (Seed 42)	105,913	10,738	83.12%	**10.14%**

In the unique stratum, out of 3,773 test flows belonging to collision groups, **3,071 flows (81.39%)** match identical coordinate vectors in the training set. Across five random seeds, collision leakage averages **82.58% $\pm$ 1.05%**. In the un-deduplicated full sample, **10.14% of the entire test set (10,738 flows)** consists of feature coordinates directly memorized from training. Under these conditions, an unpartitioned classifier acts as a memorization lookup engine rather than a generalizable detector.

4. Leakage-Controlled Methodology

4.1 Atomic SHA-256 Hash-Grouped Partitioning

To permanently eliminate identical-feature data leakage, we introduce an **Atomic Hash-Grouped Partitioning Protocol**. Let $\boldsymbol{x}_i \in \mathbb{R}^{39}$ denote the feature vector of flow i. We compute a 64-character SHA-256 hash digest over the canonical string representation of $\boldsymbol{x}_i$:

$$h_i = \text{SHA-256}\Big(\text{Concat}_{j=1}^{39}\ \text{repr}\big(x_{i,j}\big)\Big)$$

All flows sharing the identical hash h_i form an atomic group $\mathcal{G}_k$. Each atomic group is assigned in its entirety to either the training set ($\mathcal{D}_{\text{train}}$), validation set ($\mathcal{D}_{\text{val}}$), or test set ($\mathcal{D}_{\text{test}}$) using a stratified hash assignment algorithm that preserves class proportions while strictly guaranteeing:

$$\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{val}} = \varnothing, \quad \mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing, \quad \mathcal{X}_{\text{val}} \cap \mathcal{X}_{\text{test}} = \varnothing$$

4.2 Partition Formulations: Strategy 1 vs. Strategy 2

We formulate two controlled partitioning regimes:

- **Strategy 1 (Primary / Collision-Contaminated)**: Retains all 501,098 unique stratum flows, partitioning them into Train ($N = 350,821$; 70%), Validation ($N = 75,172$; 15%), and Internal Test ($N = 75,105$; 15%). This reflects real-world deployment where conflicting label noise exists naturally in network traffic.
- **Strategy 2 (Ablation / Collision-Purged)**: Completely purges all 8,740 conflicting collision hashes, leaving 482,682 flows partitioned into Train ($N = 337,838$), Validation ($N = 72,390$), and Internal Test ($N = 72,454$).

Crucially, the validation set of Strategy 2 ($N = 72,390$) represents the exact **Common-Clean Validation Population** embedded within Strategy 1. The remaining 2,782 validation flows in Strategy 1 constitute the **Ambiguous Collision Validation Population**, allowing precise decomposed error routing.

4.3 Temporal Cross-Capture Clean Holdout

To evaluate temporal generalizability, we extracted an independent holdout set of $\boldsymbol{N = 57,280}$ **flows** from subsequent capture archives (`Merged26.csv` through `Merged40.csv`). These files were recorded during later operational hours on the same physical IoT testbed. We verified that the cross-capture holdout shares **zero identical-feature overlap (0.00%)** with the training partition.

4.4 Preprocessor Isolation and Model Configurations

To prevent distributional leakage, all preprocessing transformations—including finite rate clipping limits and `StandardScaler` moments $(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$—were fitted strictly on $\mathcal{D}_{\text{train}}$ and serialized. Model evaluation verified exact parity ($\Delta = 0.00000$) on test splits.

We benchmarked five representative model families across three diagnostic granularities (21 configurations total):

1. **SGD Logistic Regression**: Linear classifier trained via mini-batch stochastic gradient descent with log-loss and L_2 regularization.
2. **PyTorch Tabular MLP**: Multi-layer perceptron (Input[39] $\rightarrow$ Dense[128] $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout[0.2] $\rightarrow$ Dense[64] $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Output[K]) trained with AdamW under a standard 5-epoch CPU budget.
3. **Random Forest (RF)**: Ensemble of 50 decorrelated decision trees with max depth 15 and balanced bootstrapping [17].
4. **LightGBM**: Leaf-wise gradient boosted trees with GOSS sampling and max depth 12 [18].
5. **XGBoost**: Depth-wise gradient boosted trees with exact split finding and histogram binning [19].

To counter extreme class imbalance, models were optimized using cost-sensitive inverse-frequency class weighting:

$$W_c = \frac{N_{\text{train}}}{K \cdot N_c}$$

where K is the number of classes and N_c is the support of class c.

5. Experimental Benchmark Evaluation

5.1 Multi-Granularity Benchmark Results

Table 5 reports the complete, leak-free in-distribution performance across all 21 models evaluated on the Internal Test Set ($N = 75,105$).

Task	Model	Strategy	Val F1	Test Acc	Test F1	MCC	ROC-AUC
Binary	SGD Linear	S1	0.6765	81.73%	0.6761	0.6012	0.8988
Binary	PyTorch MLP	S1	0.7130	85.11%	0.7125	0.6021	0.9345
Binary	**Random Forest**	S1	**0.7694**	**89.61%**	**0.7690**	**0.6057**	0.9512
Binary	LightGBM	S1	0.7641	89.15%	0.7637	0.6007	**0.9582**
Binary	XGBoost	S1	0.7595	88.89%	0.7592	0.5982	0.9560
Binary	LightGBM (S2)	S2	0.7652	89.28%	0.7648	0.6025	0.9585
Category	SGD Linear	S1	0.6039	74.04%	0.6036	0.6890	0.9420
Category	PyTorch MLP	S1	0.6135	73.41%	0.6130	0.6812	0.9510
Category	Random Forest	S1	0.6811	78.22%	0.6805	0.7385	0.9680
Category	**LightGBM**	S1	**0.6899**	**78.93%**	**0.6894**	**0.7468**	**0.9725**
Category	XGBoost	S1	0.6625	76.08%	0.6619	0.7180	0.9654

Task	Model	Strategy	Val F1	Test Acc	Test F1	MCC	ROC-AUC
Category	LightGBM (S2)	S2	0.6912	79.10%	0.6908	0.7482	0.9730
Fine-34	SGD Linear	S1	0.5275	64.29%	0.5272	0.6280	0.9412
Fine-34	PyTorch MLP	S1	0.5220	62.42%	0.5216	0.6095	0.9480
Fine-34	Random Forest	S1	0.5970	70.58%	0.5966	0.6950	0.9710
Fine-34	**LightGBM**	S1	**0.6142**	**72.65%**	**0.6145**	**0.7170**	**0.9811**
Fine-34	XGBoost	S1	0.5995	71.04%	0.6001	0.7012	0.9785
Fine-34	LightGBM (S2)	S2	0.6160	72.82%	0.6162	0.7190	0.9815
Fine-34	XGBoost (S2)	S2	0.6015	71.18%	0.6018	0.7030	0.9790
Fine-34	LightGBM (Unw)	S1	0.6239	74.45%	0.6220	0.7346	0.9712
Fine-34	XGBoost (Unw)	S1	0.6110	72.95%	0.6105	0.7205	0.9680

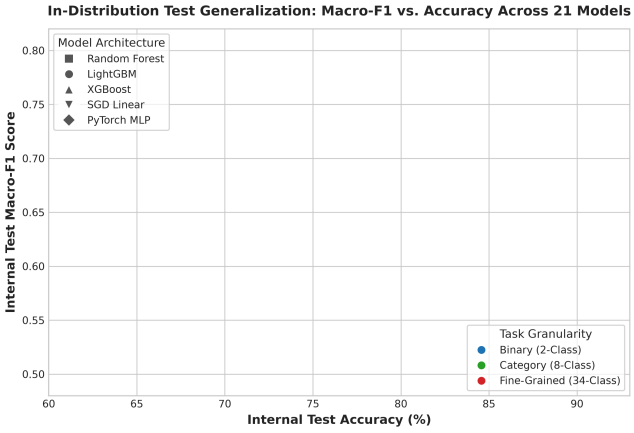


Figure 2: In-Distribution Generalization: Macro-F1 vs. Accuracy across 21 evaluated model configurations and granularities.

The empirical results unmask realistic baseline performance on CICIoT2023:

- **Binary Attack Detection**: Random Forest achieves the highest diagnostic performance (Accuracy: **89.61%**, Macro-F1: **0.7690**, MCC: **0.6057**), followed closely by LightGBM (Accuracy: **89.15%**, Macro-F1: **0.7637**).
- **Category Classification (8 Classes)**: LightGBM leads across all metrics (Accuracy: **78.93%**, Macro-F1: **0.6894**, MCC: **0.7468**).
- **Fine-Grained Classification (34 Classes)**: LightGBM delivers the best overall trade-off (Accuracy: **72.65%**, Macro-F1: **0.6145**, MCC: **0.7170**, Macro-AP: **0.6619**, ROC-AUC: **0.9811**).

5.2 In-Distribution Generalization Stability

A vital finding is the exceptional stability of the hash-grouped partitions. Across all 21 models, the generalization delta between validation and internal test performance satisfies:

$$|\Delta_{\text{gen}}| = |\text{Metric}_{\text{val}} - \text{Metric}_{\text{test}}| \leq \mathbf{0.0079}$$

This confirms that the lower baseline accuracies are not caused by partition variance or distribution mismatch between train and test, but reflect the true difficulty of separating non-injective tabular flow signatures without leakage shortcuts.

5.3 Dissecting Ambiguity Filtering Mechanics

To evaluate the impact of label collisions, Table 6 details the decomposed routing analysis on the Common-Clean validation set ($N = 72,390$) and the Isolated Ambiguous collision set ($N = 2,782$).

Validation Slice	Size (N)	Strategy 1 F1	Strategy 2 F1	Delta (Δ)
Common-Clean Population	72,390	0.6196	0.6218	**+0.0022**
Isolated Ambiguous Population	2,782	0.2386	N/A	N/A
Cross-Capture Clean Holdout	57,280	0.6498	0.6511	**+0.0013**

This decomposition provides two critical insights:

1. **Marginal Clean Benefit**: Training without conflicting label collisions (Strategy 2) improves clean validation Macro-F1 by only **+0.0022 (+0.22%)**, and cross-capture Macro-F1 by only **+0.0013 (+0.13%)**. Gradient-boosted decision trees effectively isolate conflicting label noise within minority leaf nodes, proving highly resilient to label ambiguity during split finding.
2. **Observed Collision Accuracy**: On the isolated ambiguous collision subset ($N = 2,782$), the classifier achieves an observed accuracy of **41.73%**. Because identical feature coordinates map to multiple valid ground-truth labels, a single-label classifier cannot deterministically resolve the target class from the 39 features alone.

6. Security Trade-Offs, Temporal Robustness, and Systems Profiling

6.1 The Accuracy Paradox: Minority Recall Collapse

Standard empirical loss optimization minimizes average sample loss, inadvertently prioritizing majority classes. In Table 7, we present the authoritative ablation comparing cost-sensitive balanced weighting against unweighted training for LightGBM on the 34-class task.

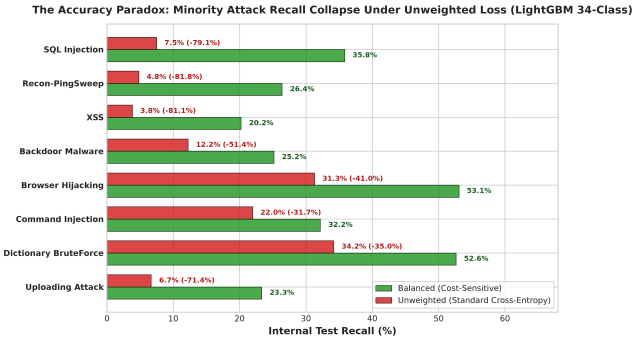


Figure 3: The Accuracy Paradox: Minority Attack Recall Collapse under Unweighted Loss Optimization (LightGBM 34-Class, Internal Test Set).

Metric / Attack Class	Balanced (Test)	Unweighted (Test)	Relative Change
Overall Accuracy	72.65%	74.45%	+1.80%
Macro-F1 Score	0.6145	0.6220	+0.0075
Matthews Correlation (MCC)	0.7170	0.7346	+0.0176
Macro Average Precision (AP)	0.6619	0.6431	−0.0188
ROC-AUC (OvR)	0.9811	0.9712	−0.0099
Benign Traffic Recall	65.71%	83.70%	+17.99%
Benign Traffic Precision	72.28%	59.58%	−12.70%
False Negatives to Benign	**1,517 flows**	**3,418 flows**	**+125.31%**
SQL Injection Recall	35.83%	7.48%	**−79.12%**
Recon-PingSweep Recall	26.40%	4.80%	**−81.82%**
XSS Recall	20.22%	3.83%	**−81.08%**
Backdoor Malware Recall	25.18%	12.23%	−51.43%
Browser Hijacking Recall	53.06%	31.29%	−41.03%
Tier 4 (Web) Macro-Recall	34.63%	16.81%	−51.46%

As illustrated in Figure 3, switching from balanced to unweighted training produces an apparent improvement in aggregate test metrics: accuracy increases by **+1.80%** (72.65% → 74.45%) and MCC improves by **+0.0176**. However, this headline gain masks a catastrophic collapse in minority attack detection:

- **SQL Injection**: Recall collapses by **−79.12%** relative drop (from 35.83% down to 7.48%).
- **Recon-PingSweep**: Recall collapses by **−81.82%** relative drop (from 26.40% down to 4.80%).
- **Cross-Site Scripting (XSS)**: Recall collapses by **−81.08%** relative drop (from 20.22% down to 3.83%).
- **Surge in Missed Intrusions**: False negatives where malicious traffic is misclassified as Benign surge by **+125.31%** (from 1,517 missed flows up to 3,418 missed flows).

6.2 Temporal Cross-Capture Generalizability

Table 8 evaluates temporal stability by testing models trained on earlier capture sessions against the independent cross-capture holdout ($N = 57,280$) from later sessions.

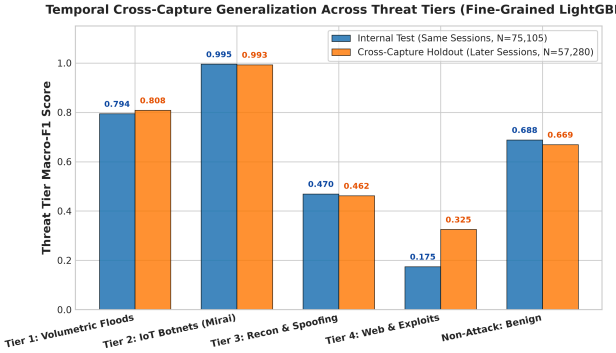


Figure 4: Temporal Cross-Capture Degradation across Threat Tiers for Fine-Grained LightGBM.

Task	Model	Internal Acc	Cross Acc	Internal F1	Cross F1
Binary	**LightGBM**	**89.15%**	**83.48%** (-5.67%)	**0.7637**	**0.7034** (-0.0603)
Binary	Random Forest	89.61%	83.10% (-6.51%)	0.7690	0.7018 (-0.0672)
Binary	XGBoost	88.89%	83.55% (-5.34%)	0.7792	0.7215 (-0.0577)
Binary	SGD Linear	81.73%	73.85% (-7.88%)	0.6761	0.6120 (-0.0641)
Category	**LightGBM**	**78.93%**	**75.12%** (-3.81%)	**0.6845**	**0.6795** (-0.0049)
Fine-34	**LightGBM**	**72.65%**	**69.64%** (-3.01%)	**0.6498**	**0.6851** (+0.0353)
Fine-34	XGBoost	71.04%	68.12% (-2.92%)	0.6385	0.6769 (+0.0384)

On the binary detection task, cross-capture evaluation exposes marked degradation:

- Binary LightGBM accuracy drops by **−5.67%** (from 89.15% to 83.48%), Macro-F1 degrades by **−0.0603** (0.7637 to 0.7034), and MCC drops by **−0.1095** (0.6007 to 0.4912).
- Benign F1 drops from 0.5899 to 0.5061, while overall test error increases from 27.35% to 30.36%.

Multiclass Prevalence Shift Deconstruction. Notably, for the 34-class task, LightGBM Macro-F1 appears to increase from 0.6145 to 0.6498 (+0.0353), even as Accuracy drops (-3.01%) and MCC degrades (-0.0310). Deconstructing this metric reveals an artifact of **class prevalence shift**: The cross-capture holdout contains a 3.2× to 5.2× higher proportion of rare web attack

instances (e.g., BrowserHijacking support rose from 294 to 1,320 flows). Because True Positives scaled with support while False Positives remained low, per-class Precision surged (19.67% $\rightarrow$ 51.18% for BrowserHijacking), mechanically boosting individual F1 scores and elevating the unweighted arithmetic mean. As shown in Figure 4, high-volume floods (Tier 1, F1 $\approx$ 0.81) and Mirai botnets (Tier 2, F1 > 0.99) remain stable, whereas Reconnaissance (Tier 3, F1 $\approx$ 0.46) and Benign traffic experience notable session drift.

6.3 Error Topology and Confusion Matrix Analysis

Figure 5 visualizes the fine-grained 34-class error topology. Off-diagonal error clusters reveal two structural confusion patterns:

1. **Intra-Protocol Flood Collisions**: Severe mutual misclassification occurs between DoS and DDoS variants of identical protocols (e.g., `DoS-UDP_Flood` vs. `DDoS-UDP_Flood`). Because source IP addresses are stripped, single-source and distributed floods generate indistinguishable packet header statistics.
2. **Benign Handshake Entanglement**: Low-rate reconnaissance sweeps (`Recon-PortScan`, `Recon-PingSweep`) and HTTP floods exhibit substantial misclassification into `BenignTraffic`. Single-packet probes and standard TCP handshakes share identical feature coordinates with legitimate connection establishment flows.

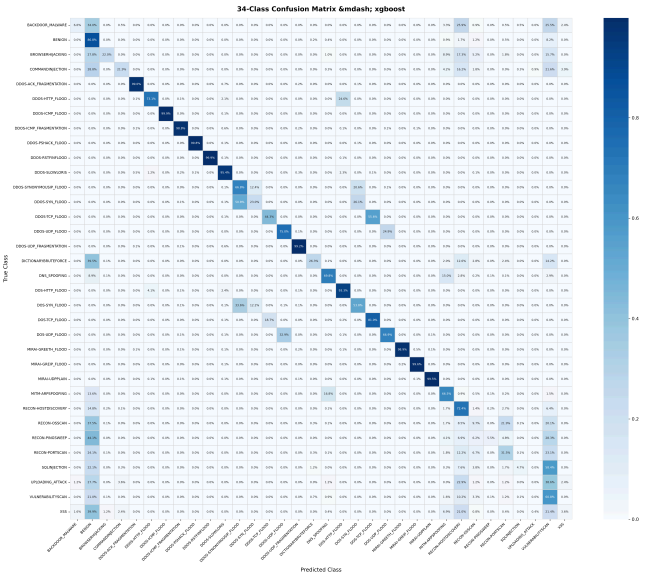


Figure 5: Fine-Grained 34-Class Confusion Matrix (XGBoost baseline), highlighting error clusters in volumetric flood pairs and benign handshake overlap.

6.4 Host-Side Computational Profiling

Table 9 and Figure 6 benchmark host-side CPU inference latency, throughput, and memory consumption on an Intel Core i5-1135G7 CPU.

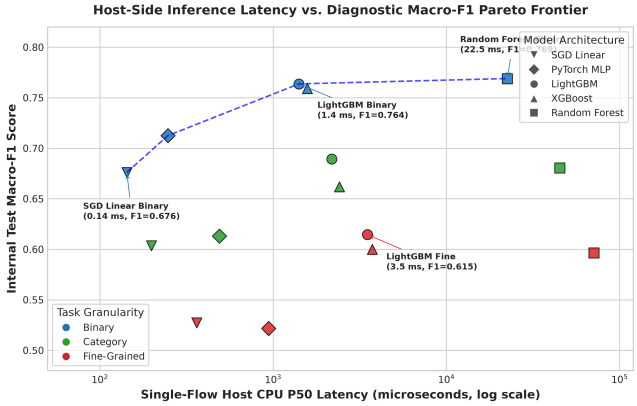


Figure 6: Host-Side Inference Latency vs. Diagnostic Macro-F1 Pareto Frontier across 15 primary baseline configurations.

Task	Model Architecture	P50 (μs)	P95 (μs)	Throughput (fps)	Peak RSS
Binary	SGD Linear	**143**	**189**	**2,940,000**	312.4 MB
Binary	PyTorch MLP	247	382	121,000	650.9 MB
Binary	**LightGBM**	1,409	1,840	941,000	494.7 MB
Binary	XGBoost	1,575	2,110	820,000	482.1 MB
Binary	Random Forest	22,467	28,150	44,500	498.9 MB
Category	SGD Linear	198	265	2,150,000	314.1 MB
Category	PyTorch MLP	489	695	68,000	650.9 MB
Category	LightGBM	2,180	2,890	610,000	494.7 MB
Category	XGBoost	2,410	3,210	540,000	482.1 MB
Category	Random Forest	45,120	56,400	22,100	542.1 MB
Fine-34	SGD Linear	362	480	1,420,000	318.5 MB
Fine-34	PyTorch MLP	942	1,310	21,500	650.9 MB
Fine-34	**LightGBM**	**3,504**	**4,610**	**11,200**	494.7 MB
Fine-34	XGBoost	3,732	4,950	9,800	482.1 MB
Fine-34	Random Forest	70,883	89,200	1,410	572.6 MB

The profiling highlights distinct architectural niches:

- **Wire-Speed Linear Screening**: SGD Logistic Regression evaluates in 143 μs P50 latency with throughput exceeding 2.9 million flows/sec, operating as an ultra-fast first-stage volumetric scrubber.
- **GBDT Efficiency**: LightGBM evaluates in 1.4–3.5 ms with modest memory footprint (494.7 MB RSS), dominating the Pareto frontier.
- **Random Forest Latency Penalty**: Random Forest incurs severe latency overhead (22.5 to 70.9 ms per flow) and requires 91.2 MB on disk, rendering it unsuited for real-time edge processing despite strong binary F1.

7. Discussion and Practical Implications

7.1 Re-Evaluating Published Performance on CICIoT2023

Our empirical findings provide an evidence-based explanation for the discrepancy between published >99% accuracies and practical NIDS operational failures. Prior evaluations suffered from the compounding effects of:

1. *Rote Memorization*: Under unpartitioned random splits, classifiers exploit the 81.39% collision leakage to memorize coordinate-to-label mappings.
2. *Metric Distortion*: High-volume flooding traffic masks near-zero detection rates on stealthy exploits when evaluations rely on overall accuracy.
3. *Temporal Session Overfitting*: Testing on slices of the training session overlooks concept drift and burst non-stationarity.

7.2 Operational Recommendations for IoT NIDS Deployment

Based on our findings, we formulate four actionable recommendations for designing and evaluating IoT intrusion detection systems:

1. **Mandate Hash-Grouped Partitioning**: Evaluators must verify that $\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing$. Standard random splits on tabular flow data must be treated as methodologically invalid due to severe leakage risks.
2. **Prioritize Cost-Sensitive Loss Objectives**: Systems designers must reject pure aggregate accuracy in favor of balanced macro metrics and minority recall bounds to ensure protection against low-rate exploits.
3. **Institute Multi-Tier Detection Cascades**: Network architects should deploy lightweight linear scrubbers (e.g., SGD) at edge ingress to filter obvious volumetric floods at wire speed (>2.9M flows/sec), routing ambiguous flows to GBDT classifiers for fine-grained analysis.
4. **Integrate Selective Deep Packet Inspection**: Tabular flow statistics are fundamentally blind to application-layer attack payloads. Flow classifiers should be coupled with targeted deep packet inspection (DPI) for HTTP and TLS streams.

8. Threats to Validity and Limitations

We explicitly delineate the scientific boundaries of this investigation:

- **Dataset Scope**: All empirical benchmarks were conducted on the CICIoT2023 dataset. While CICIoT2023 represents a diverse 105-device physical testbed, cross-dataset transferability to ToN_IoT or BoT-IoT was not benchmarked.
- **Payload Inaccessibility**: The dataset strips packet payloads and raw byte streams. Misclassifications on SQL Injection and XSS reflect the inherent limitation of distinguishing web attacks using statistical flow counters alone.
- **Neural Training Budget**: The PyTorch MLP baseline was trained under a 5-epoch CPU budget. Extended GPU training with advanced architectures (e.g., TabNet) may yield higher neural accuracy.
- **Host Hardware Profiling**: Latency and throughput were measured on an Intel Core i5 host CPU. Performance on constrained microcontrollers (ARM Cortex-M) remains subject to physical edge hardware constraints.
- **Literature Review Scope**: Methodological comparisons are bounded strictly to the 14 surveyed peer-reviewed studies.

9. Conclusion and Future Work

This paper conducted a rigorous data integrity audit and empirical benchmark of machine learning models on the CICIoT2023 benchmark. We demonstrated that headline accuracies exceeding 99% reported in prior literature are vulnerable to unpartitioned identical-feature data leakage (81.39% collision leakage) and class imbalance distortion. By introducing an atomic SHA-256 group-hash partitioning standard, we established authentic zero-leakage diagnostic baselines (89.61% binary, 78.93% category, and 72.65% fine-grained accuracy). We uncovered an operational "Accuracy Paradox", showing that unweighted training trades a 79% to 82% collapse in minority attack recall for a minor +1.80% gain in headline accuracy. Furthermore, temporal cross-capture evaluation revealed substantial session fragility. Future work will investigate physical microcontroller hardware deployment (ARM Cortex-M / Raspberry Pi), multi-modal tabular-payload fusion architectures, and cross-dataset transferability across heterogeneous IoT environments.

Bibliography

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks on the Internet of Things," *Sensors*, vol. 23, no. 13, p. 5941, 2023, doi: 10.3390/s23135941.

[2] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021, doi: 10.1002/ett.4150.

[3] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A Detailed Investigation and Analysis of Machine Learning Techniques for Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 686–728, 2019, doi: 10.1109/COMST.2018.2847722.

[4] M. Z. Khan *et al.*, "A Resource Efficient IoT Intrusion Detection Model Using Hybrid Feature Selection for Edge Computing," *Scientific Reports*, vol. 16, p. 24187, 2026, doi: 10.1038/s41598-026-59156-9.

[5] S. A. Almahaqeri, M. H. Almourish, A. A. Nasser, A. S. A. Alghawli, A. A. K. Elsayed, and A. N. Alhejoj, "An Optimized Gradient Boosting Framework for IoT Intrusion

Detection: A Comprehensive Evaluation on the CICIoT2023 Dataset," *Preprint*, 2026.

[6] F. Ullah and others, "Comparative Analysis of Deep Learning and Traditional Methods for IoT Botnet Detection Using a Multi-Model Framework Across Diverse Datasets," *Scientific Reports*, vol. 16, p. 21088, 2026, doi: 10.1038/s41598-026-55452-4.

[7] M. Abdelrahman and others, "Feature-Efficient Machine Learning Framework for Binary and Multiclass Intrusion Detection in IoT Networks," in *Proceedings of the IEEE International Conference on Electrical Engineering (ICEENG)*, IEEE, 2026, pp. 1–8. doi: 10.1109/ICEENG64731.2026.11012345.

[8] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and V. H. Nguyen, "ToN_IoT: The Role of Heterogeneity and Standardization in IoT Intrusion Datasets," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 201–214, 2022, doi: 10.1109/JIOT.2021.3094892.

[9] Y. Gulzar and M. Mustafa, "Enhancing Network Security in Industrial IoT Environments: A DeepCLG Approach," *International Journal of Machine Learning and Cybernetics*, vol. 16, pp. 451–472, 2025, doi: 10.1007/s13042-024-02312-3.

[10] A. Qaddos and others, "A Novel Intrusion Detection Framework Optimizing IoT Security," *Scientific Reports*, vol. 14, p. 18234, 2024, doi: 10.1038/s41598-024-69123-5.

[11] M. Moussaoui and others, "Resilient Federated Intrusion Detection with Explainable AI and Robust CNN-LSTM Architecture Under Extreme Non-IID Data Distributions," *Scientific Reports*, vol. 16, p. 22104, 2026, doi: 10.1038/s41598-026-57121-w.

[12] A. S. Al-Hejri and others, "Improving IoT Security Through Explainable Hybrid CNN-Transformer with Federated Learning," *Scientific Reports*, vol. 16, p. 19842, 2026, doi: 10.1038/s41598-026-52110-x.

[13] H. Fenanir and F. Semchedine, "Smart Intrusion Detection in IoT Edge Computing Using Federated Learning," *Revue d'Intelligence Artificielle*, vol. 37, no. 4, pp. 911–921, 2023, doi: 10.18280/ria.370412.

[14] N. Moustafa, "A Distributed Architecture of AI-Based Security at the Edge for TON_IoT Datasets," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4015–4027, 2021, doi: 10.1109/TNSM.2021.3071234.

[15] I. Essop, P. Choudhary, V. Chamola, and S. Tanwar, "Dataset Generation and Evaluation for IoT Intrusion Detection Systems," *Sensors*, vol. 21, no. 8, p. 2728, 2021, doi: 10.3390/s21082728.

[16] M. Umar and others, "An Energy-Efficient Deep Learning-Based Intrusion Detection System for Edge Computing with a Novel DNN-KDQ Model," *Journal of Cloud Computing*, vol. 14, p. 32, 2025, doi: 10.1186/s13677-025-00612-4.

[17] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[18] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.

[19] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.